SPM Modular Project

Module 1: Vectorization of the Partition Mapping Kernel

Student: Karol Jinet Cardona Bolanos

Mat. 702023

Academic Year 2025/2026

1 Design Choices

Mapping Function: Given an array of N 64-bit keys and a power-of-two P, we compute the partition identifier `part_id`$[i] = h($`keys`$[i]) \bmod P$ for five different hash functions. These functions represent a spectrum of algorithmic trade-offs, ranging from minimal operations with near-zero computational cost (but poor statistical properties) to complex bit-mixing formulas that guarantee mathematically uniform distributions at the expense of higher CPU overhead.

Table 1: Evaluated hash functions and their computational characteristics

Hash	Formula	Ops/key	Notes
`mask`	`key & (P-1)`	1 `AND`	Cheapest; uses low bits only
`xor_shift`	`(key ^ (key>>33)) & (P-1)`	1 `XOR`, 1 `SHF`, 1 `AND`	Light mixing, better distribution
`multiply_shift`	`(key*MULT_PRIME) >> (64-log`$_2$`P)`	1 `MUL64`, 1 `SHF`	Universal hashing (Dietzfelbinger)
`murmur_fmix64`	3× (`xor-shift` + `mul`)	2 `MUL64`, 3 `XOR-SHF`	Full avalanche mixing; highest compute cost
`fold32_mult`	Fold 64 → 32, then 32-bit `mul-shf`	1 `XOR`, 1 `MUL32`, 1 `SHF`	Enables native AVX2 8-wide multiply

Choice of Parameters (P and N). We constrain P to a power of two ($16 \leq P \leq 4096$). This replaces costly hardware division (`%`) with highly efficient bitwise `AND` (`& (P-1)`) or right-shift (`>>`) operators. Benchmarks confirm that P does not affect execution time, as expected from the constant-time nature of these operations.

To evaluate distribution quality, we computed Pearson's χ^2 and the Coefficient of Variation (CV) to measure relative load imbalance across partitions (Table 2). All CV values remain extremely low (< 0.021 even at $P = 4096$, $N = 100$M), confirming a uniformly balanced load across all hash functions. We select $P = 256$ as the reference parameter: it provides near-perfect distribution (CV ≈ 0.002) while ensuring partition metadata fits entirely within the L1 cache. For input size, $N = 10^8$ (100M keys) guarantees stable, memory-bound timings by fully amortizing loop overhead. All subsequent evaluations focus on this pair ($P = 256$, $N = 100\,\text{M}$).

Table 2: Distribution quality at $N = 100$M, $P = 256$ (values are identical across all optimization levels, as expected)

Hash	χ^2	CV
`mask`	248.6	0.001577
`xor_shift`	299.2	0.001730
`multiply_shift`	254.6	0.001596
`fold32_mult`	237.7	0.001542
`murmur_fmix64`	282.6	0.001681

The expected χ^2 for a perfectly uniform distribution over $P = 256$ bins is $P - 1 = 255$. All five functions fall within 20% of this ideal value, confirming that each is suitable for partition mapping. The fact that distribution

quality is identical across `novec`, `autovec`, and `avx2` also validates that all optimizations are semantically equivalent.

`fold32_mult` **Rationale.** Standard universal hashing relies on 64-bit multiplications, which are difficult to parallelize efficiently on AVX2 SIMD architectures since no native 64-bit vector multiplier exists. To bridge the gap between hashing quality and hardware efficiency, we engineered a function that XOR-folds the 64-bit key into 32 bits (preserving its entropy) before applying a 32-bit multiplication. This yields the lowest CV in Table 2 while targeting the native, much faster 32-bit execution units.

Correctness Verification. All implementations are validated before benchmarking through a two-level strategy. For large datasets, a FNV-1a checksum is computed over the output array and compared against an independent scalar reference. For small inputs ($N \leq 10{,}000$), a full element-by-element comparison is performed, and bounds checking ensures every partition identifier falls in $[0, P)$. All 350 CPU tests (Task 1 and Task 2) and 160 CUDA tests (Task 3) passed with zero failures, confirming bit-exact semantic equivalence across all optimization levels.

2 GCC Auto-Vectorization Evidence

Task 1 produces two distinct binaries from the same source:

- `task1_novec`: compiled with `-O3 -fno-tree-vectorize -fno-tree-slp-vectorize` (scalar baseline).
- `task1_autovec`: compiled with `-O3 -ftree-vectorize -march=native` (auto-vectorized).

Compiling with `-fopt-info-vec-optimized` produces the following report (saved to `results/autovec_report.txt`):

```
src/kernels_plain.h:9:26:  optimized: loop vectorized using 16 byte vectors
src/kernels_plain.h:19:26: optimized: loop vectorized using 16 byte vectors
src/kernels_plain.h:43:26: optimized: loop vectorized using 16 byte vectors
src/kernels_plain.h:54:26: optimized: loop vectorized using 16 byte vectors
```

GCC successfully auto-vectorized **four out of five** hot loops (`mask`, `multiply_shift`, `xor_shift`, `fold32_mult`) using **128-bit SSE vectors** (16 bytes, 2× `uint64_t` per register). The `murmur_fmix64` loop does *not* appear in the report: GCC declined to vectorize it. With only 2 `uint64_t` lanes per SSE register, vectorizing the two sequential 64-bit multiplications would require an expensive software emulation whose overhead GCC's cost model correctly estimates as unprofitable. The performance data in Section 4 confirm this: `murmur_fmix64` achieves exactly 1.00× speedup from auto-vectorization. Note also that `multiply_shift` *was* vectorized (line `19:26`), yet gains only 1.01×: the SSE2 emulation of the 64-bit multiply almost entirely cancels the benefit of parallel execution.

Key C++ techniques that enabled this compiler optimization include:

- **Pointer Aliasing Avoidance:** The `__restrict__` keyword on array pointers (`keys`, `out`) guarantees no memory overlap, enabling safe parallel SIMD loads and stores.
- **Branchless Design & Built-ins:** Loop bodies strictly avoid `if-else` branching. The `__builtin_ctzll(P)` built-in replaces a conditional log2 computation with a single hardware instruction.
- **Memory Alignment:** Data arrays are allocated on 64-byte boundaries via `aligned_alloc`, preventing cross-cacheline penalties and enabling aligned 128-bit vector loads.

3 AVX2 Intrinsics Strategy

The manual AVX2 implementation uses 256-bit registers explicitly. Depending on the mathematical requirements of the hash function, the vectorization strategy splits into two distinct paths:

64-bit Emulation Path (4 keys/cycle): Functions like `multiply_shift` and `murmur_fmix64` fundamentally require 64-bit arithmetic. Since 256-bit AVX2 registers hold four `uint64_t` values, we process 4 keys per iteration. AVX2 lacks a native $64 \times 64 \to 64$-bit multiplier, so we emulate it using `_mm256_mul_epu32` on the 32-bit halves:

$$\texttt{result} = (\texttt{a_lo} \times \texttt{b_lo}) + \big((\texttt{a_lo} \times \texttt{b_hi} + \texttt{a_hi} \times \texttt{b_lo}) \ll 32\big)$$

This requires ∼9 micro-ops per vector and incurs significant instruction overhead. The 64-bit outputs are then narrowed to 32-bit partition IDs using `_mm256_shuffle_epi32` and `_mm256_permute4x64_epi64`.

32-bit Native Path (8 keys/cycle): The `fold32_mult` function was specifically designed to exploit this path. Eight 64-bit keys are loaded, XOR-folded to 32 bits, and packed into a single 256-bit register using `_mm256_permutevar8x32_epi32`. This feeds directly into the native `_mm256_mullo_epi32` instruction, completely bypassing the 64-bit emulation. By doubling the compute width from 4 to 8 keys per cycle, this path achieves significantly higher throughput, as confirmed by the results.

4 Performance Results and Bottleneck Analysis

Benchmarks were executed on `node09` of the `spmcluster` under an exclusive SLURM allocation (`--partition=gpu-excl`), with NUMA pinning via `numactl --cpunodebind=0 --membind=0`. Timing uses the median over 15 repetitions for CPU kernels and 10 for CUDA, with standard deviation reported to confirm measurement stability.

Table 3: Throughput (Mkeys/s) at $N = 100$M, $P = 256$, and speedup relative to scalar `novec` baseline

Hash	novec	autovec	avx2	Sp. autovec	Sp. avx2
`mask`	1182.3	1575.3	1342.5	1.33×	1.14×
`xor_shift`	1060.9	1358.5	1244.1	1.28×	1.17×
`multiply_shift`	1111.7	1122.2	1066.3	1.01×	**0.96**×
`fold32_mult`	913.5	1307.8	1224.8	**1.43**×	1.34×
`murmur_fmix64`	538.4	539.5	481.0	1.00×	**0.89**×

4.1 Bottleneck Analysis: The Memory Wall

The results reveal three distinct performance tiers:

Tier 1 — Memory-bound (`mask`, `xor_shift`): These functions perform negligible computation per key. Processing 100M keys requires reading 800 MB (keys) and writing 400 MB (partition IDs). The peak CPU throughput of ∼1575 Mkeys/s translates to ∼19 GB/s effective bandwidth, approaching the DRAM ceiling of the node. SIMD vectorization yields a moderate 1.28–1.33× speedup derived primarily from reduced loop overhead and wider memory transactions, not computation savings.

Tier 2 — Compute/Memory Balanced (`fold32_mult`): `fold32_mult` benefits most from vectorization (1.43× autovec, 1.34× AVX2). The 32-bit arithmetic pairs perfectly with 8-wide SIMD, pushing throughput from 913 to 1308 Mkeys/s. That autovec (1.43×) outperforms hand-written AVX2 (1.34×) is explained by GCC generating lean 128-bit SSE code that avoids the `permutevar8x32` packing overhead incurred by the manual path.

Tier 3 — Compute-bound (`murmur_fmix64`, `multiply_shift`): Both show **no meaningful speedup or a regression** from vectorization. `murmur_fmix64` is the most striking case: GCC correctly declined to vectorize the loop (it does not appear in `autovec_report.txt`), and the hand-written AVX2 is 0.89× *slower* than scalar. The root cause is the 64-bit multiply emulation: the ∼9 micro-ops required per vector completely outweigh the gain from processing 4 keys in parallel. `multiply_shift` was vectorized by GCC (line `19:26`) but achieves only

1.01× — the SSE2 emulation cost almost entirely cancels the parallelism benefit. The manual AVX2 version compounds this to 0.96× due to additional permutation instructions.

4.2 Auto-vec vs. Hand-written AVX2

A counterintuitive result is that *auto-vectorized code outperforms hand-written AVX2 in all five cases* (Table 3). This occurs because: (1) GCC targets 128-bit SSE vectors, which are sufficient to saturate DRAM bandwidth for the memory-bound kernels and introduce less overhead for the compute-bound ones; (2) the manual 256-bit AVX2 path for 64-bit hashes incurs a multiply-emulation penalty that more than negates the wider parallelism; (3) for `fold32_mult`, the AVX2 pack/permute overhead partially offsets the 8-wide benefit that the compiler avoids with a simpler SSE2 code path. This shows that wider SIMD is not unconditionally better: it delivers gains only when the arithmetic maps *natively* to the vector width and when memory bandwidth is not the binding constraint.

5 CUDA Implementation and Correctness Verification

Task 3 maps one CUDA thread to each 64-bit key. The kernel is embarrassingly parallel: there are no thread divergence, shared memory dependencies, or synchronization barriers. With 256 threads per block and $\lceil N/256 \rceil$ blocks, the GPU is saturated immediately.

Table 4: CUDA timing breakdown ($N = 10^8$, $P = 256$, NVIDIA A30)

Hash	H2D (ms)	Kernel (ms)	D2H (ms)	Total (ms)	Ker only (Mkeys/s)
`mask`	106.53	1.49	51.34	159.37	66 908
`xor_shift`	106.56	1.50	51.27	159.33	66 789
`multiply_shift`	106.59	1.50	51.27	159.36	66 816
`fold32_mult`	106.55	1.49	51.41	159.46	66 995
`murmur_fmix64`	106.55	1.50	51.38	159.43	66 648

Kernel performance: The GPU executes any hash kernel in ∼1.50 ms, reaching ∼67 000 Mkeys/s (kernel-only throughput). This is **42**× faster than the best CPU result (`mask` autovec: 1575 Mkeys/s) and **124**× faster for `murmur_fmix64` (539 Mkeys/s on CPU). Crucially, all five hashes achieve *identical* throughput on the GPU: the High Bandwidth Memory completely flattens the compute differences that were decisive on the CPU, confirming that the GPU kernel is purely memory-bound at the device level.

PCIe transfer bottleneck: H2D and D2H transfers consume ∼107 ms and ∼51 ms respectively, totalling ∼158 ms of the ∼159 ms end-to-end time. The compute kernel accounts for less than 1% of total execution time. End-to-end throughput is therefore limited to ∼627 Mkeys/s — well below the best CPU scalar result — making GPU offloading worthwhile only when the dataset already resides in GPU memory (e.g. in a multi-kernel pipeline) or when transfers can be overlapped with computation via CUDA streams.

Correctness testing: All 160 CUDA tests (5 hashes × 4 sizes × 8 partition counts) passed with zero failures. Verification uses FNV-1a checksums for large datasets and element-by-element comparison for $N \leq 10{,}000$, with bounds checking ensuring all identifiers lie in $[0, P)$.